

TalkToData Trust Memo

Trust Memo

To: CEO, TalkToData

Subject: Should TalkToData answer questions without human review?

I would not allow TalkToData to answer questions fully unsupervised in its current form. I would require human review of the generated SQL, especially for questions tied to revenue, customer segmentation, returns, or any metric that could directly influence operational or financial decisions. The system is valuable because it collapses the time from question to answer, but the safe operating model is still "AI drafts the query, human verifies the logic."

My testing showed exactly why this review step matters. In one raw LLM test, I asked, "Which loyalty tier spends the most?" even though the schema has no `loyalty\_tier` field. Instead of refusing, the model generated valid SQL that joined `orders`, `order\_items`, `products`, and `customers`, filtered to `customers.is\_repeat = 1`, and returned the highest-spending repeat customer. The SQL ran successfully and produced a believable number, but it answered a different question from the one asked. That is more dangerous than a visible error because a decision-maker could accept the result with confidence and never realize the translation was unfaithful.

Two kinds of questions should never be answered without human checking. The first is high-impact financial questions such as revenue by segment, return-adjusted profitability, or average order value for a specific customer cohort, because a small translation error can change a real business decision. The second is ambiguous or schema-mismatched questions such as requests involving concepts the database does not actually contain, because the model may confidently invent a workaround rather than admitting uncertainty.

My understanding of trust became more concrete after building this system. At the start, I thought trust mainly meant getting a plausible answer quickly. After implementation and testing, I see trust as a workflow property: the system must show the SQL, explain what it did, block unsafe operations, validate schema references, and make mistakes visible before a human acts on the result. In practice, trust does not mean the model is always correct. It means the model's failures are catchable.